



School of Future Tech

Case Study Report

on

RewardLoop: Retail Loyalty Points & Rewards Redemption System

by

Jui Tawde

150096725149

Index

1. Introduction to the Case Study	2
2. Problem Statement / Case Background (Abstract)	2
3. Case Study Design	3
4. Methods & Algorithms Technology Applied in the Case Study	4
5. Case Study Implementation Details and Snapshots	5
6. Case Study Results and Conclusion	9
7. References	10

1. Introduction to the Case Study

Modern retail businesses depend on customer loyalty programs to encourage repeat purchases and build long-term relationships with their customers. In a loyalty program, customers earn points every time they make a purchase and can later redeem those points for rewards such as vouchers, discounts and gift packs. Managing this data by hand or in spreadsheets quickly becomes error-prone, especially when several stores and thousands of customers are involved.

This case study focuses on designing and implementing **RewardLoop**, a relational database for a Retail Loyalty Points and Rewards Redemption System. The database records customers, the stores they shop at, their purchases, the loyalty points they earn or redeem, and the catalogue of rewards available. It is implemented in PostgreSQL and queried with SQL to answer practical business questions such as a customer's current balance, total points earned and redeemed, full transaction history, and the rewards a customer can currently afford.

The case study demonstrates how an ER model is converted into normalized tables with primary and foreign keys, and how joins, aggregation and sub-queries turn raw transaction data into meaningful information.

2. Problem Statement / Case Background (Abstract)

Background

Traditional point-tracking methods, such as paper cards or flat files, cannot reliably keep a running balance, prevent a redemption that exceeds the available balance, or give a complete history of every point earned and spent. As the number of customers, stores and purchases grows, data becomes duplicated and inconsistent. A relational database enforces structure through keys and constraints and supports powerful queries over the same data.

Abstract

This case study presents the design and implementation of RewardLoop, a loyalty points and rewards redemption database, using PostgreSQL. The schema contains five related tables: Customer, Store, Purchase, LoyaltyPoint and Reward. Every purchase made by a customer at a store can generate positive loyalty point records, while every redemption creates a negative record linked to the reward that was claimed. Because balances are derived from the transaction records instead of being stored separately, the data stays consistent and auditable. SQL queries using JOIN, GROUP BY, SUM and sub-queries compute customer balances, points earned, points redeemed, transaction history with reward details, and the rewards a customer is eligible for. The results confirm that a well-designed relational schema can support all core operations of a retail loyalty program.

3. Case Study Design

The RewardLoop database is designed from the ER diagram below and follows a modular structure with five entities.

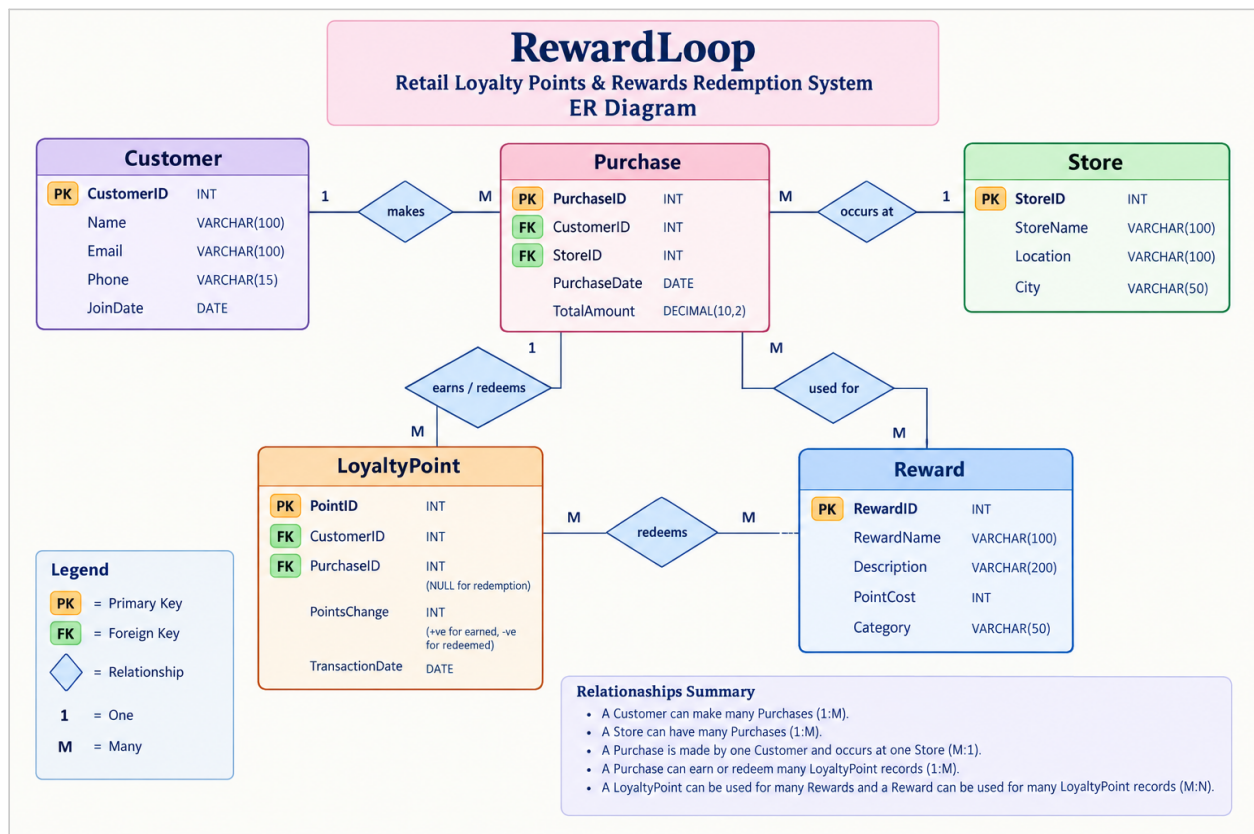


Figure 1: ER Diagram of the RewardLoop system

Entities and Their Roles

- Customer** stores the member's identity and contact details (CustomerID, FullName, Email, Phone, JoinDate).
- Store** stores the retail outlets where purchases happen (StoreID, StoreName, StoreAddress, City).
- Purchase** records every sale, linking one customer to one store with the date and amount (PurchaseID, CustomerID, StoreID, PurchaseDate, PurchaseAmount).

- **LoyaltyPoint** is the points ledger. PointsChange is positive when points are earned from a purchase and negative when points are redeemed for a reward (PointID, CustomerID, PurchaseID, RewardID, PointsChange, TransactionDate, Notes).
- **Reward** is the catalogue of redeemable items with their point cost and tier (RewardID, RewardName, Description, PointsCost, RewardTier, IsActive).

Relationships

- A Customer can make many Purchases (1:M).
- A Store can have many Purchases (1:M).
- A Purchase is made by one Customer and occurs at one Store (M:1).
- A Purchase can earn many LoyaltyPoint records (1:M); the PurchaseID is NULL for redemption records.
- A Reward can be redeemed many times through LoyaltyPoint records, and the RewardID is NULL for earning records.

Design Decisions

- Balances are never stored directly. They are calculated as the SUM of PointsChange, which avoids update anomalies.
- Each earning record points to a Purchase and each redemption record points to a Reward, so every point movement is traceable.
- Primary keys uniquely identify every row and foreign keys maintain referential integrity between the tables.

4. Methods & Algorithms Technology Applied in the Case Study

Key methods and techniques

- **Entity-Relationship modelling:** identifying entities, attributes, keys and cardinalities before creating tables.
- **Relational schema and normalization:** separate tables for customers, stores, purchases, points and rewards to remove redundancy.
- **Keys and constraints:** PRIMARY KEY and FOREIGN KEY constraints, plus data types such as INTEGER, VARCHAR, NUMERIC(10,2), TIMESTAMP and BOOLEAN.
- **Joins:** INNER JOIN to connect matching records and LEFT JOIN to keep rows that have no match (for example customers without points, or earning records without a reward).
- **Aggregation:** SUM with GROUP BY to calculate balances, points earned and points redeemed.
- **Filtering and sorting:** WHERE conditions on PointsChange and ORDER BY to rank customers.
- **Sub-queries:** a nested SELECT that returns a customer's balance, which is compared with each reward's point cost.

Technology Stack Used for the Case

- Database: PostgreSQL.
- Client tool: pgAdmin (Query Tool and Data Output viewer).

- Language: SQL (DDL for creating tables, DML for inserting data, and SELECT queries for analysis).
- Design tool: ER diagram (Chen-style notation with PK/FK markers).
- Data storage: relational tables inside a PostgreSQL database.

5. Case Study Implementation Details and Snapshots

The implementation was carried out in pgAdmin. The tables were created and populated first, and the queries below were then run on the sample data. The snapshots show the actual tables and the query outputs.

5.1 Table Structure and Sample Data

Data Output Messages Notifications					
Showing rows: 1 to 4					
	customerid [PK] integer	fullname character varying (100)	email character varying (150)	phone character varying (20)	joindate date
1	1	Aarav Mehta	aarav.mehta@example.c...	9876501001	2026-01-...
2	2	Diya Shah	diya.shah@example.com	9876501002	2026-02-...
3	3	Kabir Rao	kabir.rao@example.com	9876501003	2026-03-...
4	4	Mira Iyer	mira.iyer@example.com	9876501004	2026-04-...

Customer table: four customers with their contact details and join dates.

Stores each loyalty member. CustomerID is the primary key and is referenced by Purchase and LoyaltyPoint.

Data Output Messages Notifications				
Showing rows: 1 to 3				
Page No: 1				
	storeid [PK] integer	storename character varying (100)	storeaddress character varying (200)	city character varying (80)
1	1	RewardLoop Central	12 MG Road	Mumbai
2	2	RewardLoop Seaside	44 Palm Avenue	Navi Mumbai
3	3	RewardLoop Market	8 Station Road	Thane

Store table: three RewardLoop outlets in Mumbai, Navi Mumbai and Thane.

Stores the retail branches. StoreID is the primary key and is referenced by Purchase.

Data Output Messages Notifications					
Showing rows: 1 to 6					
Page No: 1					
	purchaseid [PK] integer	customerid integer	storeid integer	purchasedate timestamp without time zone	purchaseamount numeric (10,2)
1	101	1	1	2026-08-01 10:15:00	1200.00
2	102	1	2	2026-08-14 18:30:00	800.00
3	103	2	1	2026-08-03 13:05:00	650.00
4	104	2	3	2026-08-21 16:45:00	1400.00
5	105	3	2	2026-08-11 11:20:00	500.00
6	106	4	3	2026-08-25 19:10:00	2200.00

Purchase table: six purchases with customer, store, date and amount.

Records each sale. CustomerID and StoreID are foreign keys; PurchaseAmount is NUMERIC(10,2).

Data Output Messages Notifications							
	pointid [PK] integer	customerid integer	purchaseid integer	rewardid integer	pointschange integer	transactiondate timestamp without time zone	notes character varying (250)
1	1	1	101	[null]	120	2026-08-01 10:16:00	Points earned from purcha...
2	2	1	102	[null]	80	2026-08-14 18:31:00	Points earned from purcha...
3	3	2	103	[null]	65	2026-08-03 13:06:00	Points earned from purcha...
4	4	2	104	[null]	140	2026-08-21 16:46:00	Points earned from purcha...
5	5	3	105	[null]	50	2026-08-11 11:21:00	Points earned from purcha...
6	6	4	106	[null]	220	2026-08-25 19:11:00	Points earned from purcha...
7	7	1	[null]	1	-100	2026-08-16 12:00:00	Redeemed Coffee Voucher
8	8	2	[null]	1	-100	2026-08-22 12:00:00	Redeemed Coffee Voucher

LoyaltyPoint table: six earning rows (positive) and two redemption rows (negative).

The points ledger. Earning rows carry a PurchaseID and a NULL RewardID, while redemption rows carry a RewardID and a NULL PurchaseID.

Data Output Messages Notifications						
	rewardid [PK] integer	rewardname character varying (100)	description character varying (300)	pointscost integer	rewardtier character varying (30)	isactive boolean
1	1	Coffee Voucher	One regular hot or iced coffee	100	Bronze	true
2	2	Shopping Discount	INR 250 off an eligible purcha...	250	Silver	true
3	3	Gift Hamper	Curated retail gift hamper	500	Gold	true
4	4	Premium Member Pack	Premium seasonal gift pack	900	Platinum	true

Reward table: four rewards across Bronze, Silver, Gold and Platinum tiers.

The reward catalogue with PointsCost and RewardTier. IsActive shows whether a reward can currently be redeemed.

5.2 SQL Queries, Explanation and Output

Q1. Current balance of every customer

This query lists every customer with their current points balance. It uses a LEFT JOIN from Customer to LoyaltyPoint so that customers with no point records still appear, then SUM(PointsChange) adds earned (positive) and redeemed (negative) points together, grouped per customer. Result: Aarav 100, Diya 105, Kabir 50 and Mira 220.

```

9  -- Q1. Current balance of every customer.
10 -- Positive points are earned; negative points are redeemed.
11 SELECT Customer.CustomerID, Customer.FullName,
12        SUM(LoyaltyPoint.PointsChange) AS CurrentBalance
13 FROM Customer
14 LEFT JOIN LoyaltyPoint
15 ON Customer.CustomerID = LoyaltyPoint.CustomerID
16 GROUP BY Customer.CustomerID, Customer.FullName
17 ORDER BY Customer.CustomerID;
18

```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1

	customerid [PK] integer	fullname character varying (100)	currentbalance bigint
1	1	Aarav Mehta	100
2	2	Diya Shah	105
3	3	Kabir Rao	50
4	4	Mira Iyer	220

Snapshot: Q1 query and its output in pgAdmin

Q2. Total points earned by each customer

An INNER JOIN of Customer and LoyaltyPoint is filtered with WHERE PointsChange > 0, so only earning transactions are summed. Results are sorted in descending order of points earned. Mira Iyer leads with 220, followed by Diya 205, Aarav 200 and Kabir 50.

```

19 -- Q2. Total points earned by each customer.
20 SELECT c.CustomerID, c.FullName,
21        SUM(l.PointsChange) AS PointsEarned
22 FROM Customer c
23 JOIN LoyaltyPoint l
24 ON c.CustomerID = l.CustomerID
25 WHERE l.PointsChange > 0
26 GROUP BY c.CustomerID, c.FullName
27 ORDER BY PointsEarned desc;
28

```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1

	customerid [PK] integer	fullname character varying (100)	pointsearned bigint
1	4	Mira Iyer	220
2	2	Diya Shah	205
3	1	Aarav Mehta	200
4	3	Kabir Rao	50

Snapshot: Q2 query and its output in pgAdmin

Q3. Total points redeemed by each customer

This query keeps only rows where PointsChange < 0 and sums them with the sign reversed (-PointsChange) so redeemed points show as positive numbers. Only customers who have redeemed appear: Aarav and Diya have each redeemed 100 points.

```

29 -- Q3. Total points redeemed by each customer.
30 SELECT c.CustomerID, c.FullName,
31        SUM(-l.PointsChange) AS PointsRedeemed
32 FROM Customer c
33 JOIN LoyaltyPoint l
34 ON c.CustomerID = l.CustomerID
35 WHERE l.PointsChange < 0
36 GROUP BY c.CustomerID, c.FullName;
37

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1

	customerid [PK] integer	fullname character varying (100)	pointsredeemed bigint
1	1	Aarav Mehta	100
2	2	Diya Shah	100

Snapshot: Q3 query and its output in pgAdmin

Q5. Loyalty transaction history with reward details

Starting from LoyaltyPoint, a LEFT JOIN to Customer brings in the customer name, and a LEFT JOIN to Reward brings in the reward name and cost. Earning rows show NULL reward details, while the two redemption rows (-100 points) show Coffee Voucher with a cost of 100. All 8 transactions are returned in PointID order.

```

45 -- Q5. Show loyalty transaction history with reward details if redeemed.
46 SELECT LoyaltyPoint.PointID, Customer.FullName,
47        LoyaltyPoint.PointsChange, LoyaltyPoint.TransactionDate,
48        Reward.RewardName, Reward.PointsCost
49 FROM LoyaltyPoint
50 LEFT JOIN Customer ON LoyaltyPoint.CustomerID = Customer.CustomerID
51 LEFT JOIN Reward ON LoyaltyPoint.RewardID = Reward.RewardID
52 ORDER BY LoyaltyPoint.PointID;
53

```

Data Output Messages Notifications

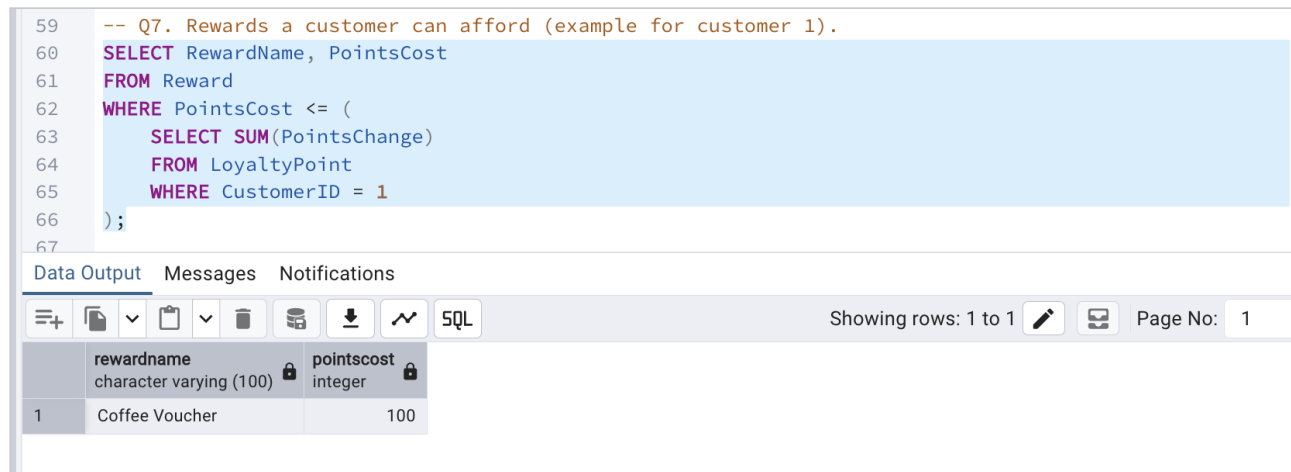
Showing rows: 1 to 8 Page No: 1

	pointid integer	fullname character varying (100)	pointschange integer	transactiondate timestamp without time zone	rewardname character varying (100)	pointscost integer
1	1	Aarav Mehta	120	2026-08-01 10:16:00	[null]	[null]
2	2	Aarav Mehta	80	2026-08-14 18:31:00	[null]	[null]
3	3	Diya Shah	65	2026-08-03 13:06:00	[null]	[null]
4	4	Diya Shah	140	2026-08-21 16:46:00	[null]	[null]
5	5	Kabir Rao	50	2026-08-11 11:21:00	[null]	[null]
6	6	Mira Iyer	220	2026-08-25 19:11:00	[null]	[null]
7	7	Aarav Mehta	-100	2026-08-16 12:00:00	Coffee Voucher	100
8	8	Diya Shah	-100	2026-08-22 12:00:00	Coffee Voucher	100

Snapshot: Q5 query and its output in pgAdmin

Q7. Rewards a customer can afford (example for customer 1)

A sub-query calculates customer 1's total balance with SUM(PointsChange). The outer query returns every reward whose PointsCost is less than or equal to that balance. Aarav has 100 points, so only the Coffee Voucher (100 points) is affordable.



```

59  -- Q7. Rewards a customer can afford (example for customer 1).
60  SELECT RewardName, PointsCost
61  FROM Reward
62  WHERE PointsCost <= (
63      SELECT SUM(PointsChange)
64      FROM LoyaltyPoint
65      WHERE CustomerID = 1
66  );
67

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1

	rewardname character varying (100)	pointscost integer
1	Coffee Voucher	100

Snapshot: Q7 query and its output in pgAdmin

6. Case Study Results and Conclusion

Findings

- Keeping every point movement as a signed row in LoyaltyPoint (positive for earning, negative for redemption) gives a complete audit trail and makes balances easy to compute.
- Customer balances match the data: Aarav 100 (200 earned, 100 redeemed), Diya 105 (205 earned, 100 redeemed), Kabir 50 and Mira 220.
- LEFT JOINS are essential for reporting, because they keep customers with no activity and earning rows that have no reward.
- Sub-queries let the system check reward eligibility directly against the current balance, for example Aarav can afford only the 100-point Coffee Voucher.
- Foreign keys between Purchase, LoyaltyPoint, Customer, Store and Reward keep the data consistent.

Conclusion

This case study demonstrates a complete workflow for building a Retail Loyalty Points and Rewards Redemption System, from ER modelling to a working PostgreSQL database. The design integrates five related tables, sample data and SQL queries covering balances, earned and redeemed points, transaction history and reward eligibility. The results show that a normalized relational design with joins, aggregation and sub-queries provides an accurate and maintainable foundation, and it can be extended with tier upgrades, point expiry rules and store-wise analytics.

7. References

- PostgreSQL Documentation: <https://www.postgresql.org/docs/>
- PostgreSQL Tutorial, Joins Between Tables:
<https://www.postgresql.org/docs/current/tutorial-join.html>
- PostgreSQL Aggregate Functions (SUM, GROUP BY):
<https://www.postgresql.org/docs/current/functions-aggregate.html>
- PostgreSQL Subquery Expressions:
<https://www.postgresql.org/docs/current/functions-subquery.html>
- pgAdmin 4 Query Tool documentation:
https://www.pgadmin.org/docs/pgadmin4/latest/query_tool.html
- Elmasri, R. and Navathe, S., Fundamentals of Database Systems, Pearson: <https://www.pearson.com/en-us/subject-catalog/p/fundamentals-of-database-systems/P200000003546/9780137502523>
- Silberschatz, A., Korth, H. and Sudarshan, S., Database System Concepts, McGraw-Hill:
<https://www.db-book.com/>